



# Collisions

## Overview

Students learn to detect and respond to things touching, in two stages. First **by hand**: shapes from `Phaser.Geom`, the tests in `Phaser.Geom.Intersects`, bounding boxes from `getBounds()`, and a circular hit area for clicks - enough to understand what any collision system does underneath, and why rectangles are unfair to round things. Then **Arcade Physics**: turning it on, giving game objects bodies, fitting bodies to pictures, and the central choice of the chapter - `collider` (separate and bounce) or `overlap` (just report) - with groups, callbacks, process callbacks, and switching collisions and bodies off. The chapter ends with a complete breakout game.

This is the first chapter that uses a physics engine. Keep to *collisions*: velocity is used, and bounce and immovable bodies are mentioned, but Chapter 9 covers movement physics (acceleration, drag, gravity, mass) properly. When students ask "how do I make it slide / fall / speed up?", point them forward.

## Prerequisites

- Chapter 1: game objects, `update()`, speeds in pixels per second
- Chapter 2: `setInteractive`, `pointerdown`, the reused-scene trap and `init()`
- Chapter 7: tweens (used for the player's flashing), and `delayedCall` from earlier chapters
- Java: casts, `instanceof`, `HashSet` - all have close TypeScript equivalents here

## Learning outcomes

Students can:

1. test whether two rectangles, two circles, or a circle and a rectangle overlap with `Phaser.Geom.Intersects`, and explain the maths behind the first two

2. explain why bounding rectangles are unfair to round objects, and give a circle a circular hit area with `setInteractive({ hitArea, hitAreaCallback })`
3. turn on Arcade Physics, give game objects bodies (`this.physics.add.image/sprite/existing`), and use `debug` to see them
4. fit a body to its picture with `setSize`, `setOffset` and `setCircle`
5. choose correctly between `collider` and `overlap`, for single objects and for groups (`group`, `staticGroup`), and use the objects passed to the callback (with `as`)
6. use a process callback to decide whether a collision counts; switch collisions off with `collider.active`, and bodies with `disableBody` / `enableBody`

## Suggested session plan (1 hour lecture + 2 hour lab)

### Lecture (1 hour)

| Time              | Activity                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0:00<br>-<br>0:05 | Play <code>ch08_breakout</code> for a minute. Ask: "how many different collisions can you name in this game? Which ones bounce, and which ones just make something happen?"                                                                                                                                                                                                                                                                           |
| 0:05<br>-<br>0:20 | Slides 3-7: geometry shapes, the two tests (draw the four rectangle comparisons on the board), <code>getBounds</code> , testing every pair, unfair corners, a circular hit area. Demo <code>ch08_hand_made_collisions</code>                                                                                                                                                                                                                          |
| 0:20<br>-<br>0:25 | <b>Live demo:</b> in the shapes project, arrange two circles corner to corner, then press SPACE. "Would you accept that as a hit in a game?"                                                                                                                                                                                                                                                                                                          |
| 0:25<br>-<br>0:35 | Slides 8-10: Arcade Physics, bodies, <code>debug</code> , <code>setSize</code> / <code>setOffset</code> / <code>setCircle</code> . Turn <code>debug: true</code> on in <code>ch08_collect_and_avoid</code> and rebuild                                                                                                                                                                                                                                |
| 0:35<br>-<br>0:50 | Slides 11-15: collider vs overlap, groups, callbacks and <code>as</code> , switching off. <b>Live demo:</b> replace the stars' overlap with <code>this.physics.add.collider(this.player, this.stars)</code> - no callback - and play: the player shoves the stars, which slide away across the screen. Then put the callback back on the collider: the star is still collected, after a one-frame bump. Ask why the overlap is still the right choice |
| 0:50<br>-<br>1:00 | Slides 16-18: breakout - the process callback and the paddle angle. Slides 19-20: summary and challenges                                                                                                                                                                                                                                                                                                                                              |

## Lab (2 hours)

| Time        | Activity                                                                                                          |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| 0:00 - 0:20 | Build and play all three projects. Answer the "What to look at" questions in each README                          |
| 0:20 - 0:40 | Challenge 1 (debug key) and 2 (hover highlight)                                                                   |
| 0:40 - 1:20 | Challenges 3 and 4 (tough bricks, wide paddle)                                                                    |
| 1:20 - 2:00 | Challenges 5 and 6 (glass bricks, fair triangles) for those who are ready; the extension ideas below for the rest |

## Key points to stress

- a collision test is **just geometry**. Arcade Physics does the same tests as Part 1 - it only does them for you, efficiently, and responds to them. Students who understand Part 1 debug Part 2 much faster
- **the body is not the picture**. Every "it hit me but it didn't touch me" bug is a body that does not match what the player sees. The first move is always `debug: true`
- **collider or overlap** is a design decision: *should these two push each other apart?* Walls, crates, bricks, paddles: yes. Pickups, hazards, trigger zones: no
- colliders are set up **once**, in `create()`, not every frame in `update()`
- an overlap callback fires **every frame** the two things touch - students must design for that (invulnerability, disabling the pickup's body, a flag)
- move bodies with **velocity**, not by setting `x / y` - otherwise the physics engine cannot stop them at walls. (The paddle following the mouse is the exception, and the chapter says why it is safe)
- **static bodies do not follow their game objects**: `refreshBody()` after moving or scaling one

# Common problems and errors

| What students see                                                                                                              | Cause                                                                                                                                                | Fix                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TypeError: Cannot read properties of undefined (reading 'add') at create , on this.physics.add...</code>                 | no <code>physics</code> section in the game config                                                                                                   | add <code>physics: { d "arcade", arcade: } }</code> to <code>main.ts</code>                                                                        |
| <code>TypeError: Cannot read properties of null (reading 'setSize') in a class that extends Phaser.Physics.Arcade.Image</code> | extending the Arcade class does not make a body; <code>scene.physics.add.existing(this)</code> is missing (or comes after the <code>setSize</code> ) | call <code>scene.physics.ad</code> in the constructor, k method                                                                                    |
| <code>TS2339 [ERROR]: Property 'disableBody' does not exist on type 'Body   StaticBody   Tile   GameObjectWithBody'.</code>    | using a callback parameter directly                                                                                                                  | cast it: <code>star as Phaser.Physics.Ar</code>                                                                                                    |
| <code>TS2531 [ERROR]: Object is possibly 'null'. on this.player.body.velocity</code>                                           | a game object's <code>body</code> is typed <code>...   null</code>                                                                                   | <code>this.player.body</code> use the object's ow ( <code>setVelocity</code> )                                                                     |
| The player bumps or pushes a pickup instead of collecting it                                                                   | <code>collider</code> used where <code>overlap</code> was meant                                                                                      | use <code>overlap</code> for pi                                                                                                                    |
| All lives lost at once when touching an enemy                                                                                  | the overlap callback runs every frame they touch                                                                                                     | invulnerability: <code>col false</code> for a while, c callback                                                                                    |
| The player stops short of a crate, or walks into its edge                                                                      | a static body scaled or moved without <code>refreshBody()</code> , or a body the wrong size                                                          | <code>refreshBody()</code> ; t                                                                                                                     |
| The body is offset from the sprite in the wrong direction                                                                      | <code>setOffset</code> / <code>setCircle</code> offsets measured from the centre by mistake                                                          | they are measured <b>top-left</b> corner                                                                                                           |
| A click just outside a circle still picks it up                                                                                | default rectangular hit area                                                                                                                         | <code>hitArea: new Phaser.Geom.Circ1</code><br><code>hitAreaCallback:</code><br><code>Phaser.Geom.Circ1</code><br>the centre is <code>(r, r</code> |

| What students see                                                                | Cause                                                                                                                                                                                                                                                       | Fix                                                                              |
|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| A process callback seems to do nothing - everything still collides               | the callback forgot <code>return</code> . Phaser only skips a collision when it gets exactly <code>false</code> , and <code>undefined</code> is not <code>false</code> . TypeScript does not warn: Phaser types the callback as returning <code>void</code> | <code>return</code> the boolean                                                  |
| With debug on, outlines of bricks already knocked out are still drawn (breakout) | the bricks were removed with <code>disableBody</code> : a disabled <i>static</i> body no longer collides, but the debug drawing still shows it                                                                                                              | harmless, but confusing bricks that will never be removed in the chapter project |
| After a restart, debug drawing is off again (challenge 1)                        | each scene start gets a new physics world                                                                                                                                                                                                                   | expected; store the registry if it should be                                     |

## Discussion questions

- The shapes project tests 15 pairs a frame. How many would a bullet-hell game with 500 bullets and one player need, if bullets never hit each other? (500 - collisions *between* groups are far cheaper than every object against every other.) Why does `collider(enemies, enemies)` cost more than `collider(player, enemies)` ?
- For each of these, collider or overlap: a door you walk through to change room; a spring that launches you; a moving platform; a checkpoint flag; a bullet hitting a wall; a bullet hitting an enemy
- The enemy body is smaller than the triangle. Is it fair? Who benefits? Should an enemy bullet's body be smaller or bigger than its picture? What about a coin's?
- Stars are disabled and re-enabled; bricks are destroyed. When is each better? (Reuse vs. gone for good; Chapter 14 comes back to this as object pools.)
- The paddle sets its `x` directly when following the mouse, but the player never does. Why is it safe for the paddle but not for the player?

---

## Extension ideas

- in the shapes project, add a `Phaser.Geom.Triangle` shape and the three new tests it needs ( `RectangleToTriangle` , `TriangleToCircle` , `TriangleToTriangle` )
- breakout: a second ball ("multi-ball") - one more collider line per ball, or put the balls in a group so the existing colliders just work
- collect and avoid: a bomb pickup that clears every enemy on screen ( `this.enemies.clear(true, true)` )
- collision **categories** ( `this.physics.nextCategory()` , `setCollisionCategory` , `setCollidesWith` ) as an alternative to choosing which pairs get colliders
- breakout: levels read from an array of strings ( `"XX..XX"` ), a preview of tilemaps (Chapter 10)

---

## Assessment ideas

- Practical: "add spikes to `ch08_collect_and_avoid` - a static group of hazards that cost a life when touched, but that enemies bounce off"
- Code reading: show a scene with `this.physics.add.collider(...)` inside `update()` and ask what is wrong and what happens as the game runs (a new collider every frame - it gets slower and slower, and callbacks fire many times)
- Short answer: explain the difference between a collide callback and a process callback, with an example of a game rule that needs a process callback
- Paper exercise: given two circles `(100, 100, r 30)` and `(150, 140, r 25)` , do they overlap? Do their bounding boxes? (Distance  $64.0 > 55$ , so no; the boxes overlap: yes)

---

## Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

# Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

---

## 1. Debug key

**Project:** `solutions/ch08_challenge_1_debug_key` (from `ch08_collect_and_avoid`)

**Goal:** find the physics world, and learn that `debug` is a setting you can change while the game runs.

`src/scenes/GameScene.ts`

```
this.input.keyboard!.on("keydown-D", () => {
  this.toggleDebug();
});
...
private toggleDebug(): void {
  const world = this.physics.world;
  if (!world.debugGraphic) {
    world.createDebugGraphic();
  } else {
    world.drawDebug = !world.drawDebug;
    world.debugGraphic.clear();
  }
  this.debugText.setVisible(world.drawDebug);
}
```

With `debug: false` in the config, the world has no `Graphics` to draw on, so the first press calls `createDebugGraphic()`, which makes one and turns `drawDebug` on. After that, `drawDebug` is flipped. The `clear()` matters: the world clears and redraws the debug graphic every frame *while* `drawDebug` is on - switch it off without clearing, and the last frame's outlines stay frozen on screen.

**Look for:** the frozen-outlines bug (very common - ask the student to toggle it off while things are moving); the "DEBUG" label kept in step with `world.drawDebug`, not with a separate flag that can drift. A student who sets `debug: true` in the config and then only

flips `drawDebug` has a valid answer, but the game no longer starts with it off. After game over and SPACE the debug drawing is off again: each scene start makes a new physics world. Tested with the harness: D, D, D, then a game over and restart.

---

## 2. Hover highlight

**Project:** `solutions/ch08_challenge_2_hover_highlight` (from `ch08_hand_made_collisions`)

**Goal:** point-in-shape tests by hand, with the right test for each shape.

`src/geometry.ts`

```
export function contains(shape: DragShape, x: number, y: number): boolean {
  if (shape instanceof Phaser.GameObjects.Arc) {
    return Phaser.Geom.Circle.Contains(circleOf(shape), x, y);
  }
  return Phaser.Geom.Rectangle.Contains(shape.getBounds(), x, y);
}
```

`src/scenes/ShapesScene.ts` (called from `update()`)

```

private showHover(): void {
  const pointer = this.input.activePointer;
  let hovered: DragShape | undefined = undefined;

  for (const shape of this.shapes) {
    if (contains(shape, pointer.x, pointer.y)) {
      if (hovered === undefined || this.children.getIndex(shape) >
this.children.getIndex(hovered)) {
        hovered = shape;
      }
    }
  }

  for (const shape of this.shapes) {
    if (shape === hovered) {
      shape.setStrokeStyle(HOVER_OUTLINE_WIDTH, HOVER_OUTLINE_COLOUR);
    } else {
      shape.setStrokeStyle(); // no line width: no outline
    }
  }
}

```

`this.input.activePointer` is the mouse (or the last finger), in game coordinates. When shapes overlap, only the one drawn on top is outlined - the one a click would pick up - found with its index in the scene's display list.

**Look for:** a circle test for circles (the challenge says the corner must not count - test it by hovering just outside a circle, diagonally); outlines removed from the other shapes. Highlighting *every* shape under the pointer is an acceptable simpler answer. Doing it in "pointermove" instead of `update()` works while the mouse moves, but misses a shape being dragged *under* a still pointer. Tested with the harness: centre of the big circle (outlined), its box corner (nothing), a box, and a circle dragged on top of a box (the circle wins).

### 3. Tough bricks

**Project:** `solutions/ch08_challenge_3_tough_bricks` (from `ch08_breakout`)

**Goal:** keep state on each game object, and let a collision callback decide what a hit means.

`src/scenes/BreakoutScene.ts` - when the bricks are made

```
const tough = row < TOUGH_ROWS;
brick.setData(HITS_LEFT, tough ? TOUGH_HITS : 1);
brick.setData(POINTS, tough ? TOUGH_POINTS : POINTS_PER_BRICK);
```

and in `hitBrick()`

```
const hitsLeft = (brick.getData(HITS_LEFT) as number) - 1;
if (hitsLeft > 0) {
  // still standing: remember the hit, and look cracked
  brick.setData(HITS_LEFT, hitsLeft);
  brick.setTint(CRACKED_COLOUR);
  return;
}

// CHALLENGE 3: read the points BEFORE destroy() - destroying a game object
// clears its data too
this.score += brick.getData(POINTS) as number; // was POINTS_PER_BRICK

// gone for good: destroy() removes the brick from the scene, its body from
// the physics world,
// and the brick from the group
brick.destroy();
```

Every game object has a **data manager**: `setData(name, value)` and `getData(name)` store values of your own on the object itself - no subclass needed. The ball still bounces off a tough brick on the first hit, because the collider does the bounce before the callback runs.

**Look for:** the order of `getData` and `destroy()` - reading the points after destroying the brick gives `undefined`, and the score becomes `NaN` - this happened while writing the solution, and only playing it showed the problem. A `Brick` subclass with a `hitsLeft` field is an equally good, more OO answer - it needs `classType: Brick` on the group, or bricks made with `new Brick(...)` and `this.bricks.add(brick)`. Tested with the harness: the ball dropped onto a top-row brick twice - first hit leaves it (cracked colour, `hitsLeft 1`), second removes it and scores 30.

## 4. Wide paddle

**Project:** `solutions/ch08_challenge_4_wide_paddle` (from `ch08_breakout`)

**Goal:** a new dynamic group created during play, an overlap, a timer, and a body that changes size.

`src/scenes/BreakoutScene.ts`

```
this.powerUps = this.physics.add.group();
this.physics.add.overlap(this.paddle, this.powerUps, (_paddle, powerUp) => {
    // the paddle was passed first, so the power-up is the second object
    (powerUp as Phaser.Physics.Arcade.Sprite).destroy();
    this.sound.play(PADDLE_SOUND);
    this.paddle.widen(WIDE_FACTOR, WIDE_TIME);
});
...
private dropPowerUp(x: number, y: number): void {
    const powerUp = this.powerUps.create(x, y, POWER_UP_KEY) as
    Phaser.Physics.Arcade.Sprite;
    powerUp.setVelocityY(POWER_UP_SPEED);
}
```

`src/objects/Paddle.ts`

```
public widen(factor: number, duration: number): void {
    this.setScale(factor, 1);
    this.narrowTimer?.remove(); // `?.` - only call remove() if there is a
    timer
    this.narrowTimer = this.scene.time.delayedCall(duration, () => {
        this.setScale(1, 1);
    });
}
```

`hitBrick()` calls `dropPowerUp(brick.x, brick.y)` one time in five, and `update()` destroys any power-up that has fallen below the screen (looping over a copy of the group's list, because `destroy()` removes items from it). The picture is `gem.png`, copied from the asset library.

**Look for:**

- an **overlap**, not a collider - with a collider the paddle (immovable) would bounce the gem away
- the body grows with the picture: a *dynamic* body follows its game object's scale by itself. The harness confirms the paddle's body goes from 104 to 156 pixels wide and back. (A static body would need `refreshBody()` )
- `moveTo` and `bounceOff` already use `displayWidth` , so the wider paddle clamps and aims correctly with no other changes - worth pointing out as a reward for not hard-coding the width
- a second power-up caught while wide should not end the effect early: removing the old timer handles that. A common bug is stacking timers, so the first one to fire shrinks the paddle while the second power-up's time is still running
- missed power-ups destroyed, not left falling for ever

Tested with the harness: a power-up caught (scale 1.5, body 156), one missed (destroyed), a second catch 6 seconds later keeps the paddle wide past the first timer's end, and it returns to normal 10 seconds after the second catch.

---

## 5. Glass bricks

**Project:** `solutions/ch08_challenge_5_glass_bricks` (from `ch08_breakout` )

**Goal:** a process callback that does something, then cancels the collision.

`src/scenes/BreakoutScene.ts`

```

this.physics.add.collider(
  this.ball,
  this.bricks,
  (_ball, brick) => {
    // Phaser's types allow for a body or a tile too; we passed the bricks
    group, so it is one
    // of its members - and a group makes Arcade Sprites unless told
    otherwise
    this.hitBrick(brick as Phaser.Physics.Arcade.Sprite);
  },
  // CHALLENGE 5: a process callback. Glass is smashed HERE, and false tells
  Phaser not to
  // collide - so no bounce, and the collide callback above is not called
  for glass. Every
  // other brick answers true, and collides as before.
  (_ball, brick) => {
    const hit = brick as Phaser.Physics.Arcade.Sprite;
    if (hit.getData(GLASS) === true) {
      this.hitBrick(hit);
      return false;
    }
    return true;
  },
);

```

and, when the bricks are made, the glass row is marked:

```

if (row === GLASS_ROW) {
  brick.setTint(GLASS_COLOUR);
  brick.setAlpha(GLASS_ALPHA);
  brick.setData(GLASS, true);
}

```

The process callback runs *before* separation. Returning `false` means no separation, no bounce and no collide callback - so the glass must be smashed inside the process callback itself.

### Look for:

- `return true` for ordinary bricks. Without it the callback returns `undefined` for them, which Phaser treats as "collide" (it only skips on exactly `false`) - so this bug is invisible here, but it is sloppy and TypeScript does not catch it

- smashing in the collide callback instead: when the process callback returns `false` the collide callback never runs, so glass is never smashed - the ball passes through it and it stays there for ever. (And if the process callback returns `true` for glass, the ball bounces)
- an alternative some students find: an `overlap` for glass and a `collider` for the rest, with the bricks split into two groups. Perfectly good - and worth comparing with the process callback version

Tested with the harness: with the bricks above and below a glass brick removed, a ball fired up through it smashes it and keeps going up (velocity still -420); an ordinary brick still bounces the ball back.

---

## 6. Fair triangles

**Project:** `solutions/ch08_challenge_6_fair_triangles` (from `ch08_collect_and_avoid`)

**Goal:** combine both halves of the chapter: physics finds the candidates, geometry decides.

`src/scenes/GameScene.ts`

```

this.enemyOverlap = this.physics.add.overlap(
  this.player,
  this.enemies,
  () => {
    this.hurtPlayer();
  },
  (_player, enemy) => {
    return this.touchesTriangle(enemy as Phaser.Physics.Arcade.Sprite);
  },
);
...
// CHALLENGE 6: does the player's body really touch this enemy's red
// triangle?
private touchesTriangle(enemy: Phaser.Physics.Arcade.Sprite): boolean {
  // the enemy picture's top-left corner, in the game (its origin is its
  // centre)
  const left = enemy.x - enemy.width / 2;
  const top = enemy.y - enemy.height / 2;
  const [ax, ay, bx, by, cx, cy] = TRIANGLE_POINTS;
  const triangle = new Phaser.Geom.Triangle(left + ax, top + ay, left + bx,
  top + by, left + cx, top + cy);

  // the player's body, as a rectangle. body.x and body.y are its top-left
  // corner
  const body = this.player.body!;
  const box = new Phaser.Geom.Rectangle(body.x, body.y, body.width,
  body.height);

  return Phaser.Geom.Intersects.RectangleToTriangle(box, triangle);
}

```

and each enemy's body now covers the whole triangle:

```

enemy.setSize(40, 38);
enemy.setOffset(4, 4);

```

The body is a cheap first test ("close enough to be worth checking"); the process callback does the exact test, and only a real touch reaches `hurtPlayer()`. This two-stage idea - a quick **broad phase**, then an exact **narrow phase** - is how every physics engine works.

**Look for:**

- the body made **bigger** to cover the whole triangle. Keeping the small body means real touches near the triangle's bottom corners are never even checked - the most common half-right answer
- the test in the process callback, not in the overlap callback. In the overlap callback it can still work ( `if (!this.touchesTriangle(...)) return;` ), which is acceptable; the process callback is the idiomatic place
- the triangle built in game coordinates from the picture's top-left corner ( `x - width / 2` ), not from its centre
- the player's shape: its body rectangle is the fair choice (it matches the blob). Using `this.player.getBounds()` - the whole 48 x 48 picture - reintroduces unfair corners

Tested with the harness, against the chapter project: an enemy placed so only an empty corner of its picture overlaps the player costs no life in either; one placed so the player touches the triangle's bottom-left corner costs a life in the solution but not in the chapter project (the small body misses it); a full overlap costs a life in both.

# Slides

## Chapter 8

### Collisions

When this touches that



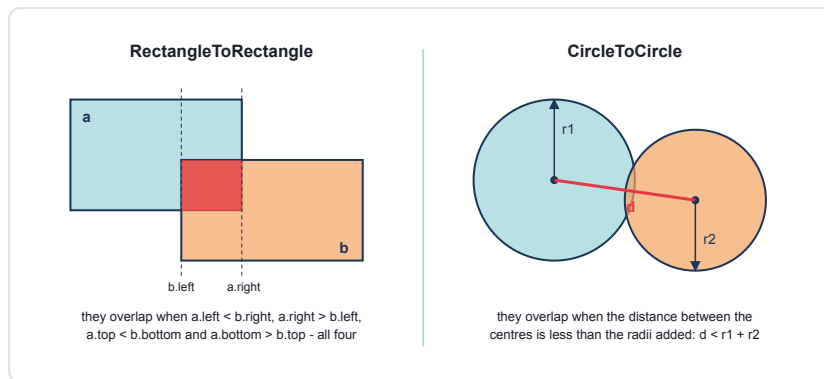
1 / 20

### Today

- collisions **by hand**: `Phaser.Geom` shapes and `Intersects` tests
- why rectangles are **unfair** to round things
- **Arcade Physics**: bodies, and `debug` to see them
- fitting a body to its picture
- `collider` or `overlap` ? Groups against groups
- process callbacks, and switching collisions off
- a complete **breakout**

2 / 20

## Shapes made of numbers



Phaser.Geom.Rectangle, Circle, Triangle ... - never drawn.

Phaser.Geom.Intersects.RectangleToRectangle(a, b), CircleToCircle, CircleToRectangle

3 / 20

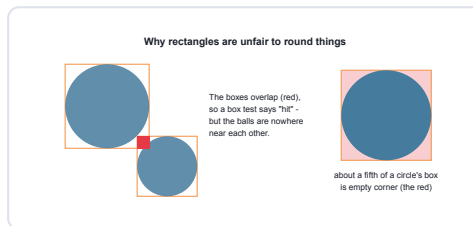
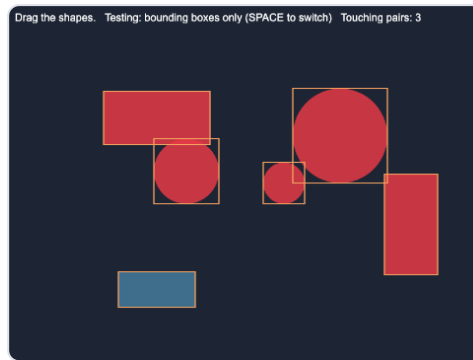
## Which test?

```
if (a instanceof Phaser.GameObjects.Arc && b instanceof Phaser.GameObjects.Arc) {
    return intersects.CircleToCircle(circleOf(a), circleOf(b));
}
if (a instanceof Phaser.GameObjects.Arc) {
    return intersects.CircleToRectangle(circleOf(a), b.getBounds());
}
if (b instanceof Phaser.GameObjects.Arc) {
    return intersects.CircleToRectangle(circleOf(b), a.getBounds());
}
return intersects.RectangleToRectangle(a.getBounds(), b.getBounds());
```

- `getBounds()` - any game object's **bounding box**
- `instanceof` **narrows** the type: `a.radius` is allowed inside the `if`
- every pair, once: `for (j = i + 1 ...)` - 6 shapes, 15 pairs; 100 shapes, 4,950

4 / 20

## Unfair corners



Round things: test **circles**. Boxy things: rectangles are exact.

5 / 20

## Clicking a circle

```
circle.setInteractive({
  hitArea: new Phaser.Geom.Circle(radius, radius, radius),
  hitAreaCallback: Phaser.Geom.Circle.Contains,
  draggable: true,
  useHandCursor: true,
});
```

- default hit area: the **rectangle** - a click in the corner picks it up
- hit area coordinates: (0, 0) is the **top-left**, so the centre is (radius, radius)
- Contains(shape, x, y) - on every Phaser.Geom class

6 / 20

## Try it: Hand-made Collisions

- build, serve, drag the shapes together
- press SPACE: bounding boxes only. Find a "hit" that is not a hit
- click just outside a circle, diagonally. Does it move?



7 / 20

## Arcade Physics

```
// main.ts
physics: {
  default: "arcade",
  arcade: {
    debug: false,    // true: draw every body
  },
},
```

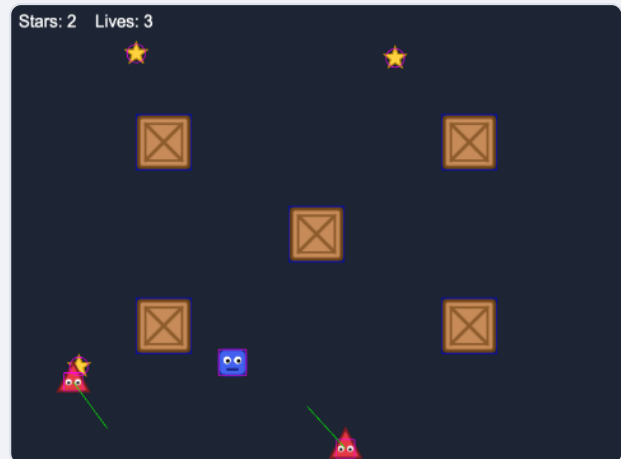
```
this.physics.add.image(x, y, key);    // a new Image, with a body
this.physics.add.sprite(x, y, key);   // a new Sprite, with a body
this.physics.add.existing(gameObject); // a body for an object you have
```

8 / 20

## Bodies - and seeing them

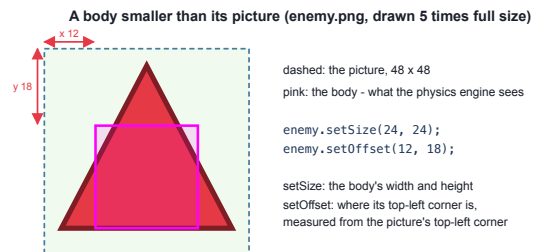
```
export class Player extends
Phaser.Physics.Arcade.Image {
  constructor(scene: Phaser.Scene, x:
number, y: number) {
    super(scene, x, y, PLAYER_KEY);
    scene.add.existing(this);
    scene.physics.add.existing(this);
```

- move with **velocity**:  
    `this.setVelocity(speedX, speedY)`
- pink: dynamic bodies; blue: static



9 / 20

## Fitting the body



dashed: the picture, 48 x 48  
pink: the body - what the physics engine sees

```
enemy.setSize(24, 24);  
enemy.setOffset(12, 18);
```

setSize: the body's width and height  
setOffset: where its top-left corner is,  
measured from the picture's top-left corner

```
star.setCircle(12, 4, 5); // radius, then offset from the top-left
```

10 / 20

## Collider or overlap?

**this.physics.add.collider(a, b)**

1. a moves into b

2. Phaser pushes them apart



separates them, changes their velocities (bounce), THEN calls your callback

walls, crates, bricks, the paddle

**this.physics.add.overlap(a, b)**

a moves into b - and nothing is pushed



only calls your callback, every frame that they still touch

pickups, hazards, trigger areas

Should these two push each other apart?

11 / 20

## All the collisions - set up once, in `create()`

```
this.physics.add.collider(this.player, this.crates);
this.physics.add.collider(this.enemies, this.crates);
this.physics.add.collider(this.enemies, this.enemies); // a group against itself

this.physics.add.overlap(this.player, this.stars, (_player, star) => {
  this.collectStar(star as Phaser.Physics.Arcade.Sprite);
});
this.enemyOverlap = this.physics.add.overlap(this.player, this.enemies, () => {
  this.hurtPlayer();
});
```

No collision code in `update()` at all

12 / 20

## Groups

```
this.crates = this.physics.add.staticGroup();    // never move
...
this.enemies = this.physics.add.group({          // dynamic
  bounceX: 1,
  bounceY: 1,
  collideWorldBounds: true,
});
...
const crate = this.crates.create(x, y, CRATE_KEY) as Phaser.Physics.Arcade.Sprite;
crate.setScale(CRATE_SCALE);
crate.refreshBody();    // a static body does not follow its game object
```

13 / 20

## Callbacks, casts, and switching off

TS2339: Property 'disableBody' does not exist on type 'Body | StaticBody | Tile | GameobjectWithBody' -> star as Phaser.Physics.Arcade.Sprite

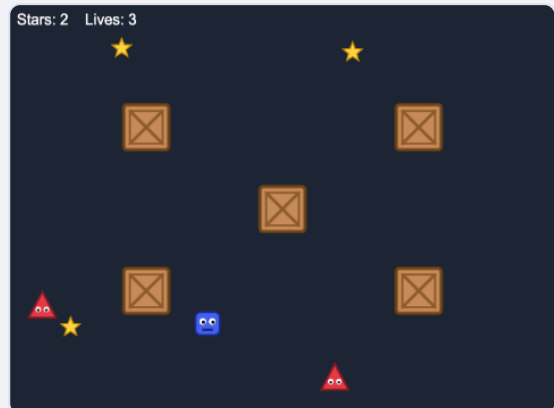
```
star.disableBody(true, true);    // body off, hidden
...
star.enableBody(true, spot.x, spot.y, true, true);    // back, somewhere else
...
this.enemyOverlap.active = false;    // stop checking for a while
...
this.enemyOverlap.active = true;    // ...and start again
```

An overlap fires **every frame** they touch

14 / 20

## Try it: Collect and Avoid

- arrow keys; stars in, triangles out
- set `debug: true` in `main.ts`, rebuild
- replace the stars' overlap with `this.physics.add.collider(this.player, this.stars)`. What happens?



15 / 20

## Try it: Breakout

- paddle: dynamic, **immovable**
- ball: `setCircle`, `setBounce(1)`, world bounds
- bricks: a **static group**, destroyed when hit

```
// no bottom edge: the ball falls out  
this.physics.world.setBoundsCollision(true, true, true,  
false);
```



16 / 20

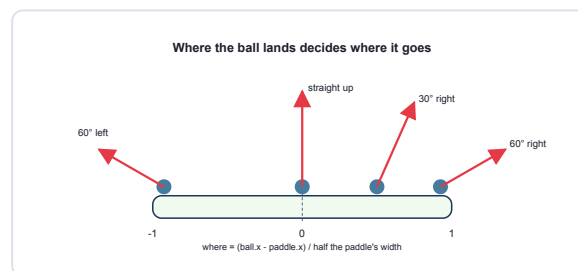
## A process callback

```
this.physics.add.collider(  
  this.ball,  
  this.paddle,  
  () => {  
    this.ball.bounceOff(this.paddle);  
    this.sound.play(PADDLE_SOUND);  
  },  
  () => {  
    return this.ball.isFalling(); // false: no collision at all  
  },  
);
```

Runs **first**. Only exactly `false` cancels - forget the `return` and everything collides

17 / 20

## Aiming the ball



```
const where = Phaser.Math.Clamp((this.x - paddle.x) / halfWidth, -1, 1);  
this.setDirection(where * MAX_BOUNCE_ANGLE);
```

18 / 20

## Summary

- `Phaser.Geom + Intersects ; getBounds() ;` circles for round things
- physics config; `this.physics.add.image / sprite / existing ; debug`
- `setSize + setOffset , setCircle` - from the **top-left**
- `collider` separates; `overlap` reports. Once, in `create()`
- `group / staticGroup ; refreshBody() ;` cast callback objects with `as`
- process callbacks; `collider.active ; disableBody / enableBody ; destroy()`

19 / 20

## Challenges

1. **Debug key** - D toggles debug drawing
2. **Hover highlight** - outline the shape under the pointer, fairly
3. **Tough bricks** - the top row takes two hits
4. **Wide paddle** - a falling power-up widens the paddle for 10 s
5. **Glass bricks** - smashed, but no bounce
6. **Fair triangles** - only a real touch of the triangle counts

Next: **Chapter 9 - 2D physics**

20 / 20